Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

Nivel 1: Entorno e ingesta híbrida (Code-First)

Escenario: el cuadro de mando financiero tarda demasiado en cargar y su coste es inaceptablemente alto. Se ha detectado que el problema crítico está en los informes que cruzan ventas con regiones geográficas.

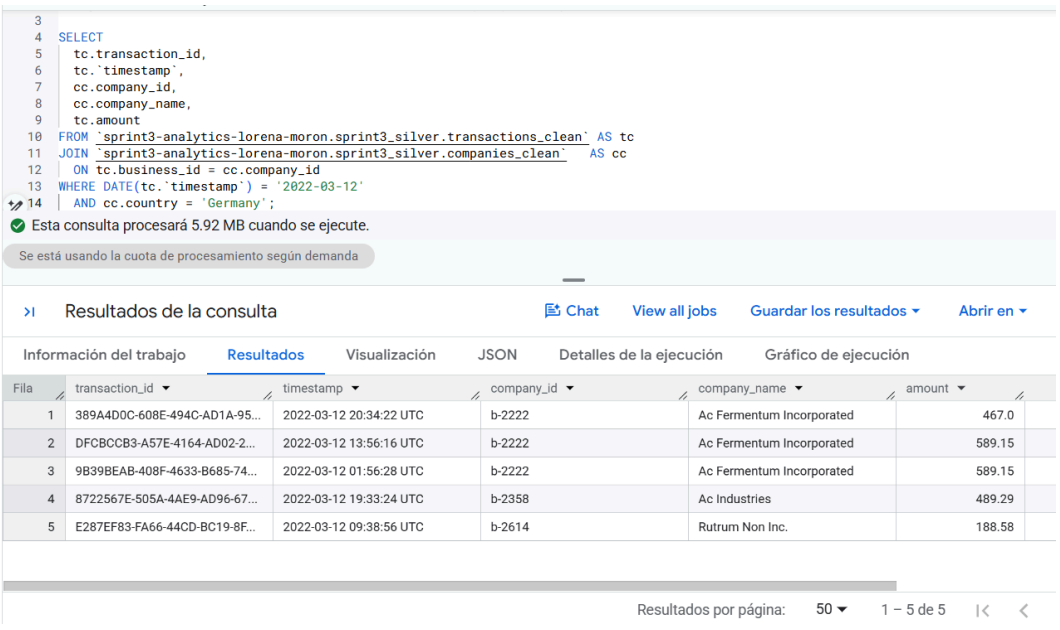
Ejercicio 1: Consulta sobre una tabla no optimizada (diagnóstico)

El Country Manager de Alemania necesita revisar con urgencia las transacciones del 12 de marzo de 2022.

🧩 Tarea:
Escribe la consulta que une (JOIN) transacciones y compañías.
Filtra los resultados por la fecha indicada y por el país "Germany".
Sin ejecutar la consulta, realiza un Dry Run (auditoría de costes).
👀 Observación: fíjate en que BigQuery lee casi toda la tabla aunque solo le pidas un día (Full Table Scan).

👉 Entrega:

Antes de ejecutar la consulta, el validador indica que procesará 5,92 MB. Al cambiar la fecha del filtro o al eliminar el WHERE, el volumen apenas varía. Esto demuestra que se produce un Full Table Scan: BigQuery almacena los datos por columnas y, como la tabla no está particionada, no sabe dónde están las filas del 12/03/2022. Por tanto, lee todos los valores de las columnas utilizadas en la consulta (en SELECT, JOIN y WHERE) y aplica los filtros después. Pedir un solo día cuesta prácticamente lo mismo que pedir todo el histórico.



```
3
4  SELECT
5    tc.transaction_id,
6    tc.`timestamp`,
7    cc.company_id,
8    cc.company_name,
9    tc.amount
10 FROM `sprint3-analytics-lorena-moron.sprint3_silver.transactions_clean` AS tc
11 JOIN `sprint3-analytics-lorena-moron.sprint3_silver.companies_clean`   AS cc
12   ON tc.business_id = cc.company_id
13 WHERE DATE(tc.`timestamp`) = '2022-03-12'
14   AND cc.country = 'Germany';
```

Esta consulta procesará 5.92 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta | Chat | View all jobs | Guardar los resultados | Abrir en

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	transaction_id	timestamp	company_id	company_name	amount
1	389A4D0C-608E-494C-AD1A-95...	2022-03-12 20:34:22 UTC	b-2222	Ac Fermentum Incorporated	467.0
2	DFCBCCB3-A57E-4164-AD02-2...	2022-03-12 13:56:16 UTC	b-2222	Ac Fermentum Incorporated	589.15
3	9B39BEAB-408F-4633-B685-74...	2022-03-12 01:56:28 UTC	b-2222	Ac Fermentum Incorporated	589.15
4	8722567E-505A-4AE9-AD96-67...	2022-03-12 19:33:24 UTC	b-2358	Ac Industries	489.29
5	E287EF83-FA66-44CD-BC19-8F...	2022-03-12 09:38:56 UTC	b-2614	Rutrum Non Inc.	188.58

Resultados por página: 50 | 1 – 5 de 5

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

Ejercicio 2: Rediseño y optimización del almacenamiento (Partition & Cluster)

Escenario: tras el diagnóstico del ejercicio anterior, hemos confirmado que filtrar la tabla de transacciones por fecha o por empresa provoca un Full Table Scan (lectura completa de la tabla), lo cual es ineficiente. Para solucionarlo en la capa de consumo (Gold), materializaremos una versión optimizada de la tabla de hechos.

Problema del Sandbox: como usamos un entorno gratuito sin tarjeta de crédito, BigQuery borra automáticamente los datos de las particiones con más de 60 días de antigüedad. Dado que nuestros datos históricos son antiguos, si particionamos la tabla directamente, acabará vacía. Para evitarlo, lo resolveremos en dos pasos: primero actualizaremos automáticamente las fechas de los datos y, después, aplicaremos la optimización física.

🧩 Tarea:

Paso 1: Generación de datos recientes (Mocking Data)
Crea una tabla intermedia llamada sprint3_silver.transactions_recent a partir de la tabla sprint3_silver.transactions_clean. El objetivo es conservar todas las columnas, pero sustituir el timestamp original por uno nuevo, generado aleatoriamente para que caiga dentro de los últimos 50 días.

Pista técnica: para no escribir todas las columnas a mano, puedes usar SELECT * EXCEPT(timestamp). Para calcular la fecha aleatoria, tendrás que restar una cantidad de días al instante actual. Te resultarán muy útiles las funciones TIMESTAMP_SUB() y CURRENT_TIMESTAMP(), junto con la combinación CAST(RAND() * 50 AS INT64) para generar los días de desfase.

Paso 2: Creación de la tabla optimizada (Partitioning & Clustering)
Ahora crea tu tabla definitiva sprint3_gold.fact_transactions_optimized a partir de los datos recientes que acabas de generar en transactions_recent. Debes construir la sentencia DDL que configure la tabla con las siguientes estrategias de almacenamiento físico:

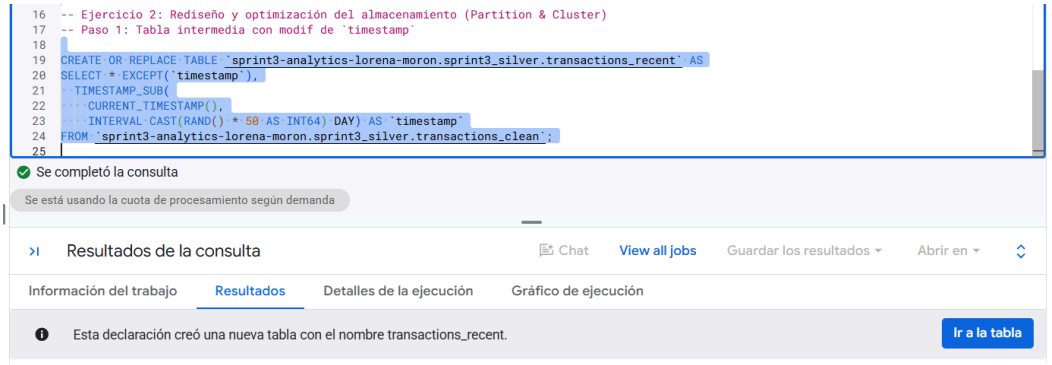
Particionado (Partitioning): divide la tabla según la fecha del campo DATE(timestamp). Así, las consultas que filtren por día (WHERE date = ...) sólo leerán la partición necesaria.
Clustering: ordena los datos dentro de cada partición por business_id. Esto acelerará drásticamente los filtros por compañía y los cruces (JOIN) con la dimensión de empresas.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

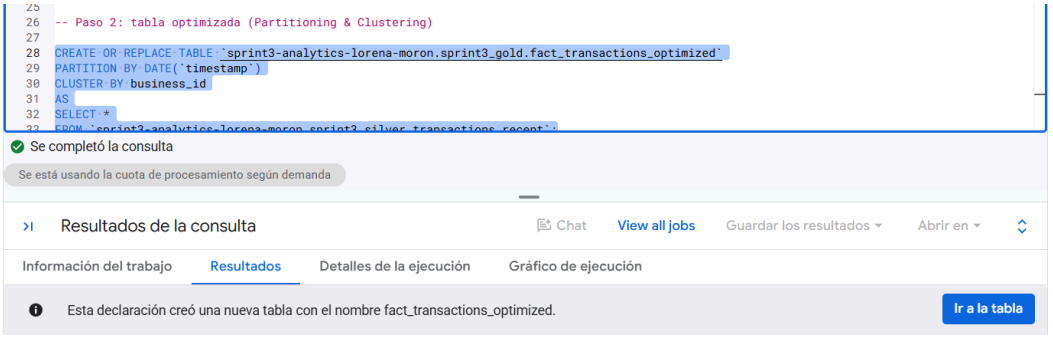
Realizado por: Lorena Morón

👉 Entrega:

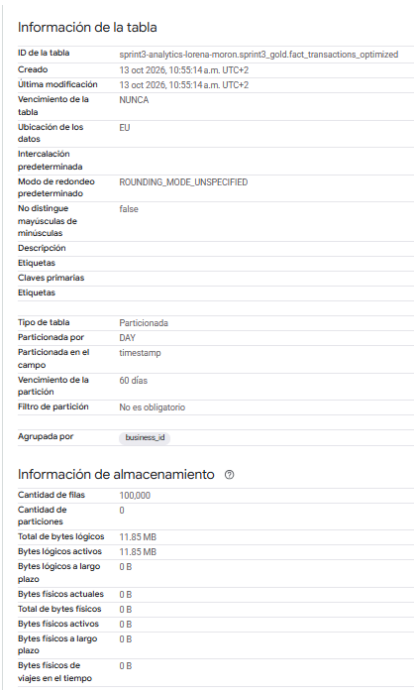
El código SQL utilizado en el Paso 1.



El código DDL (CREATE OR REPLACE TABLE...) utilizado en el Paso 2.



Una captura de pantalla de los "Detalles" de la nueva tabla fact_transactions_optimized en la que se compruebe que está correctamente particionada y agrupada en clústeres.



Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

Ejercicio 3: La prueba de fuego (Benchmark)

Escenario: ya has creado la tabla sprint3_gold.fact_transactions_optimized con las estrategias de particionado y clustering. Ahora toca demostrar al equipo la mejora de rendimiento y el ahorro de costes conseguidos. Simularemos una petición típica de negocio: "Queremos ver todas las transacciones de los últimos 30 días".

🧩 Tarea:
El objetivo es claro y directo: aplicar exactamente la misma consulta a dos tablas distintas y comparar su coste computacional.

Construye una consulta SQL que seleccione todas las columnas (SELECT *) y filtre los datos de los últimos 30 días.

⚠️ NO ejecutes todavía las consultas. En este ejercicio solo usaremos el validador de BigQuery (Dry Run, en la esquina superior derecha del editor) para ver gratis los bytes procesados.

Haz la comparativa en dos pasos:

Paso 1 (tabla no optimizada):
Apunta tu consulta a la tabla sprint3_silver.transactions_recent. Observa el validador y haz una captura de pantalla de los bytes que procesaría.
Paso 2 (tabla optimizada):
Sin cambiar nada de la lógica de la consulta, modifica solo el FROM para que apunte a la tabla particionada sprint3_gold.fact_transactions_optimized. Observa cómo cambia el validador y haz la segunda captura.

👉 Entrega:

El código SQL base que has diseñado.
Las dos capturas de pantalla en las que se vea la diferencia de "Bytes procesados" en el validador.

```
37  -- Paso 1: codigo para tabla NO optimizada)
38  SELECT *
39  FROM `sprint3-analytics-lorena-moron.sprint3_silver.transactions_recent`
40  WHERE DATE(`timestamp`) BETWEEN DATE_SUB(CURRENT_DATE(), INTERVAL 29 DAY)
41                          AND CURRENT_DATE();
```

Esta consulta procesará 11.85 MB cuando se ejecute.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

```
-- Paso 2: codigo para tabla OPTIMIZADA

SELECT *
FROM `sprint3-analytics-lorena-moron.sprint3_gold.fact_transactions_optimized`
WHERE DATE(`timestamp`) BETWEEN DATE_SUB(CURRENT_DATE(), INTERVAL 29 DAY)
                        AND CURRENT_DATE();
```

Esta consulta procesará 7 MB cuando se ejecute.

Un breve texto con el cálculo del porcentaje de ahorro conseguido (comparando los bytes del Paso 1 con los del Paso 2).

Consulta en la tabla NO OPTIMIZADA (transactions_recent), se esperan procesar 11,85 MB
Consulta en la tabla OPTIMIZADA (fact_transactions_optimized), se esperan procesar 7 MB

Ahorro: (11,85 − 7) / 11,85 × 100 = 40,93 %.

La tabla particionada descarta los días fuera del rango y lee solo 30 de los 51 días disponibles. El ahorro es moderado porque los datos simulados solo cubren unos 51 días. Con varios años de histórico, superaría el 95 %. El clustering no interviene aquí, porque la consulta no filtra por business_id, con lo cual el ahorro solo proviene del particionado.

Ejercicio 4: Smart Caching (vistas materializadas)

Escenario: el Director General tiene un cuadro de mando que muestra las "Ventas totales por día". Él (y otros 50 managers) actualizan este gráfico constantemente.

Problema: si usas una consulta normal o una vista estándar, cada vez que alguien actualiza la página, BigQuery vuelve a calcular la suma de millones de filas. Es lento y estás pagando una y otra vez por el mismo cálculo.

Solución: usar una vista materializada. Esta tecnología guarda el resultado del cálculo en caché y solo lo actualiza cuando entran datos nuevos (de forma incremental).

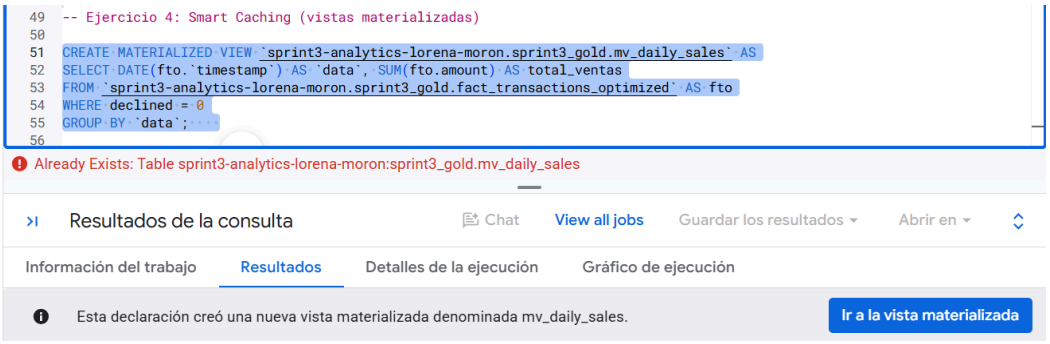
🧩 Tarea:
Crea una vista materializada llamada sprint3_gold.mv_daily_sales que muestre las ventas totales por día.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

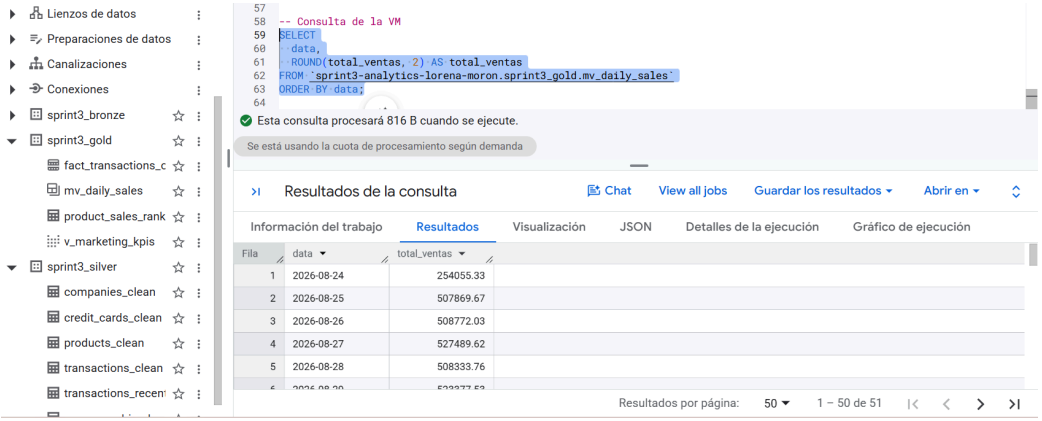
👉 Entrega:

El código para crear la vista materializada.



Nota: "El aviso 'Already Exists' corresponde a una segunda ejecución; la vista se creó correctamente en la primera, como se ve en el mensaje inferior."

Una consulta que muestre la vista creada y una captura de pantalla en la que se vea en conjunto la vista materializada, la consulta ejecutada y los bytes procesados.



Nivel 2: SQL analítico avanzado

Escenario: la dirección empieza a plantear preguntas que no se resuelven con un simple GROUP BY. Necesitamos análisis de tendencias y de comportamiento de usuario.

Ejercicio 1: Perfilado de clientes VIP (métricas agregadas con CTE)

Escenario: el equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos como "VIP" a quienes tienen un gasto acumulado superior a 500 €. Necesitan un informe que muestre, para cada VIP, su nombre, su contacto y su patrón de compra: cuántas veces ha comprado, cuánto gasta de media y cuál fue su compra récord.

🧩 Tarea:

Crea una CTE llamada VIP_Stats que agrupe por usuario y calcule:

El gasto total (SUM).

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

El número de transacciones (COUNT).
El ticket medio (AVG), redondeado a 2 decimales.
La compra máxima (MAX).
Filtro: conserva solo a los usuarios cuyo gasto total sea > 500.
Cruza la CTE con users_combined para obtener los datos personales.

Requisitos de salida:
Columnas: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat. Ordenado por total_gastat de forma descendente.

```sql
-- Nivel 2: SQL analítico avanzado
-- Ejercicio 1: Perfilado de clientes VIP (métricas agregadas con CTE)

WITH vip_stats AS(
  SELECT
    fto.user_id,
    ROUND(SUM(fto.amount), 2) AS total_gasto,
    COUNT(fto.transaction_id) AS n_transacciones,
    ROUND(AVG(fto.amount), 2) AS ticket_medio,
    MAX(fto.amount) AS compra_maxima
  FROM `sprint3-analytics-lorena-moron.sprint3_gold.fact_transactions_optimized` AS fto
  WHERE fto.declined = 0
  GROUP BY fto.user_id
  HAVING SUM(fto.amount) > 500)

  SELECT vs.user_id,
  CONCAT(uc.name, ' ', uc.surname) AS nombre_cliente,
    uc.email,
    vs.n_transacciones,
    vs.ticket_medio,
    vs.compra_maxima,
    vs.total_gasto
  FROM vip_stats AS vs
  JOIN `sprint3-analytics-lorena-moron.sprint3_silver.users_combined` AS uc
    ON vs.user_id = uc.user_id
  ORDER BY vs.total_gasto DESC;
```

Esta consulta procesará 6.17 MB cuando se ejecute.

Resultados de la consulta

Chat | View all jobs | Guardar los resultados | Abrir en

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	user_id	nombre_cliente	email	n_transacciones	ticket_medio	compra_maxima	total_gasto
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwfll Phylgnmhnd	fwfll.phylgnmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9
9	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47
10	317	Qikcvc Nvfcifqdh	qikcvc.nvfcifqdh@example.com	54	351.69	600.77	18991.45
11	388	Rbjupnac Cfjykdx	rbjupnac.cfjykdx@example.com	62	297.3	479.03	18432.84
12	319	Fxtg Eedgl	fxtg.eedgl@example.com	62	286.62	576.22	17770.39
13	443	Qimzjbfg Cvpdvtz	qimzjbfg.cvpdvtz@example.com	75	230.81	524.33	17310.42
14	393	Nxbq Bmbzxiihu	nxbq.bmbzxiihu@example.com	76	220.09	534.22	16726.47
15	437	Kxnl Rzhpnaz	kxnl.rzhpnaz@example.com	50	325.21	572.37	16260.46
16	183	Germane Whitehead	malesuada.malesuada.integer...	76	194.98	624.64	14818.17
17	402	Fpymtvae Kvuvznta	fpymtvae.kvuvznta@example.c...	76	188.35	737.25	14314.3
18	120	Solomon Blake	in.at.pede@hotmail.edu	45	318.0	626.48	14310.21
19	345	Kcglr Ddaqewaoh	kcglr.ddaqewaoh@example.com	52	271.12	728.35	14098.42
20	15	Noelani Patrick	sem.magna@google.com	49	280.45	576.57	13741.97
21	389	Apmudcg Jvzeqkk	apmudcg.jvzeqkk@example.com	52	263.47	515.43	13700.39
22	472	Ccgz Eoynyholwn	ccgz.eoynyholwn@example.com	45	296.12	727.55	13325.38

Resultados por página: 50 | 1 – 50 de 5000

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

Ejercicio 2: Análisis de tendencias (Window Functions sobre vistas)

Escenario: la Dirección Financiera quiere monitorizar la "velocidad de ventas" diaria. Necesita un informe que compare el rendimiento de cada día con el del día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).
📈 Estrategia de optimización: para este cálculo NO debes ir a la tabla de hechos original ni volver a calcular sumas. Como buen Data Engineer, debes usar la vista materializada sprint3_gold.mv_daily_sales que creaste en el ejercicio anterior. Así garantizas que el análisis de tendencias sea instantáneo y no consuma recursos de cómputo innecesarios.

🧩 Tarea: crea una consulta sobre la vista materializada que genere un informe con estas 4 columnas:

Data: la fecha de venta.
Vendes_Avui: el total vendido ese día.
Vendes_Ahir: el total vendido el día anterior (usando funciones de ventana).
Diff_Percentual: el porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales.

💡 Pista técnica: usa la función LAG(campo) OVER (ORDER BY data) para leer el valor de la fila anterior sin necesidad de hacer JOIN complejos.

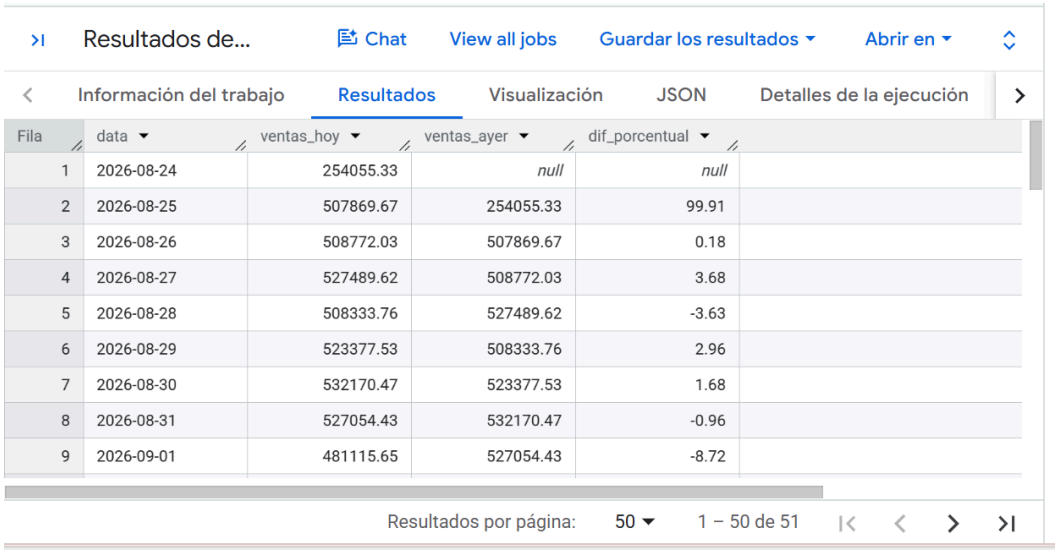
```sql
-- Ejercicio 2: Análisis de tendencias (Window Functions sobre vistas)

WITH ventas_diarias AS (
  SELECT
    ds.data,
    ds.total_ventas                                  AS ventas_hoy,
    LAG(ds.total_ventas) OVER (ORDER BY ds.data)     AS ventas_ayer
  FROM `sprint3-analytics-lorena-moron.sprint3_gold.mv_daily_sales` AS ds
)
SELECT
  vd.data,
  ROUND(vd.ventas_hoy, 2)  AS ventas_hoy,
  ROUND(vd.ventas_ayer, 2) AS ventas_ayer,
  ROUND(SAFE_DIVIDE(vd.ventas_hoy - vd.ventas_ayer, vd.ventas_ayer) * 100, 2) AS dif_porcentual
FROM ventas_diarias AS vd
ORDER BY vd.data;
```

Esta consulta procesará 816 B cuando se ejecute.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón


Resultados de... Chat View all jobs Guardar los resultados Abrir en

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución

Fila	data	ventas_hoy	ventas_ayer	dif_porcentual
1	2026-08-24	254055.33	null	null
2	2026-08-25	507869.67	254055.33	99.91
3	2026-08-26	508772.03	507869.67	0.18
4	2026-08-27	527489.62	508772.03	3.68
5	2026-08-28	508333.76	527489.62	-3.63
6	2026-08-29	523377.53	508333.76	2.96
7	2026-08-30	532170.47	523377.53	1.68
8	2026-08-31	527054.43	532170.47	-0.96
9	2026-09-01	481115.65	527054.43	-8.72

Resultados por página: 50 1 – 50 de 51

Ejercicio 3: Totales acumulados (Running Totals sobre vistas)

Escenario: el Director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuánto llevamos facturado en total desde el inicio del año hasta hoy (YTD, Year To Date).

🧩 Tarea: usando la vista materializada mv_daily_sales (para no volver a calcular las agregaciones base), genera un informe con tres columnas:
Data.
Ventas del día: redondeadas a 2 decimales.
Ventas acumuladas YTD: una suma progresiva de las ventas que debe reiniciarse cada 1 de enero. El resultado final debe estar redondeado a 2 decimales.

💡 Pista técnica: para conseguir el reinicio anual, necesitas dividir la ventana con PARTITION BY EXTRACT(YEAR FROM data). La estructura completa sería:
SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

👉 Entrega:

```sql
-- Ejercicio 3: Totales acumulados (Running Totals sobre vistas)

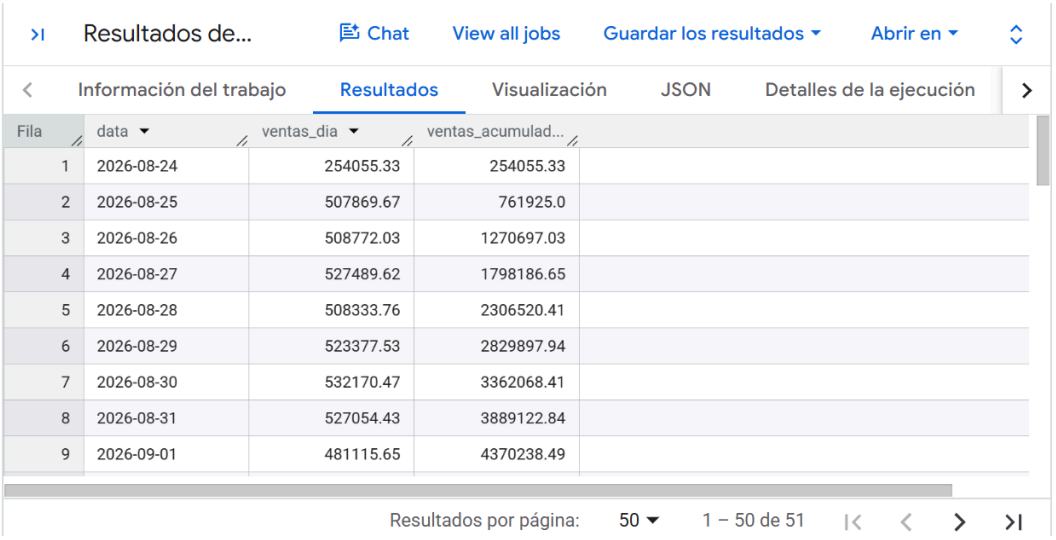
SELECT
  ds.data,
  ROUND(ds.total_ventas, 2) AS ventas_dia,
  ROUND(
    SUM(ds.total_ventas) OVER (
      PARTITION BY EXTRACT(YEAR FROM ds.data)
      ORDER BY ds.data
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ), 2) AS ventas_acumuladas_ytd
FROM `sprint3-analytics-lorena-moron.sprint3_gold.mv_daily_sales` AS ds
ORDER BY ds.data;

```

Esta consulta procesará 816 B cuando se ejecute.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón



Fila	data	ventas_dia	ventas_acumulad...
1	2026-08-24	254055.33	254055.33
2	2026-08-25	507869.67	761925.0
3	2026-08-26	508772.03	1270697.03
4	2026-08-27	527489.62	1798186.65
5	2026-08-28	508333.76	2306520.41
6	2026-08-29	523377.53	2829897.94
7	2026-08-30	532170.47	3362068.41
8	2026-08-31	527054.43	3889122.84
9	2026-09-01	481115.65	4370238.49

Nota: la cláusula PARTITION BY EXTRACT(YEAR FROM ds.data) divide los datos por año, de modo que el total acumulado se reinicia automáticamente cada 1 de enero. En los resultados actuales este reinicio no se aprecia porque todas las fechas simuladas pertenecen a 2026. Aun así, la consulta está preparada para funcionar correctamente cuando la tabla contenga datos de varios años.

Ejercicio 4: Fidelización y valor del cliente (filtrado avanzado)

Escenario: el equipo de Marketing lanza la campaña "A la tercera va la vencida". Quiere enviar un regalo a los usuarios que hayan completado su tercera transacción. Sin embargo, el presupuesto del regalo no es fijo: depende de la "calidad" del cliente en sus inicios. Necesitan conocer el ticket medio de esas 3 primeras compras para decidir si envían un detalle básico o un regalo premium.

🧩 Tarea: genera un listado de los usuarios que han alcanzado (o superado) su tercera compra, mostrando:
Los datos del usuario (user_id, nom_complet, email).
La fecha y el importe exacto de la 3.ª compra.
Media de las 3 primeras: el gasto medio de sus transacciones 1, 2 y 3.

💡 Pista técnica:
Debes usar la función ROW_NUMBER() para establecer el orden cronológico de las compras.
Debes usar la cláusula QUALIFY para filtrar.
Reto de optimización: busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el historial es costoso. Intenta filtrar las filas necesarias antes de realizar la agregación final.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

👉 Entrega:

```sql
-- Ejercicio 4: Fidelización y valor del cliente (filtrado avanzado)

WITH primeras_compras AS (
  SELECT
    tc.user_id,
    tc.`timestamp`,
    tc.amount,
    ROW_NUMBER() OVER (
      PARTITION BY tc.user_id
      ORDER BY tc.`timestamp`, tc.transaction_id
    ) AS n_compra
  FROM `sprint3-analytics-lorena-moron.sprint3_silver.transactions_clean` AS t
  WHERE tc.declined = 0
  QUALIFY n_compra <= 3
),
metricas AS (
  SELECT
    pc.user_id,
    MAX(IF(pc.n_compra = 3, pc.`timestamp`, NULL))    AS fecha_3a_compra,
    MAX(IF(pc.n_compra = 3, pc.amount, NULL))         AS importe_3a_compra,
    ROUND(AVG(pc.amount), 2)                          AS media_3_primeras
  FROM primeras_compras AS pc
  GROUP BY pc.user_id
  HAVING COUNT(*) = 3
)
SELECT
  m.user_id,
  CONCAT(uc.name, ' ', uc.surname) AS nombre_completo,
  uc.email,
  m.fecha_3a_compra,
  m.importe_3a_compra,
  m.media_3_primeras
FROM metricas AS m
JOIN `sprint3-analytics-lorena-moron.sprint3_silver.users_combined` AS uc
  ON m.user_id = uc.user_id
ORDER BY m.media_3_primeras DESC;
```

✅ Esta consulta procesará 6.94 MB cuando se ejecute.

Según los resultados obtenidos, los 5.000 clientes (el total de clientes que visualizamos en la tabla 'users_combined') han alcanzado su tercera compra. Por tanto, el criterio de "tercera compra" no segmenta: todos serían elegibles. La variable que permite diferenciarlos es la media de gasto de las tres primeras compras, que Marketing puede usar para decidir entre un detalle básico y un regalo premium.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

Resultados de la consulta

Chat | View all jobs | Guardar los resultados | Abrir en

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	user_id	nombre_completo	email	fecha_3a_compra	importe_3a_com...	media_3_primeras
1	444	Eklf Srqdlsxkk	eklf.srqdlsxkk@example.com	2016-05-12 20:56:06 UTC	674.66	581.87
2	2124	Ltbjxm Zlielgna	ltbjxm.zlielgna@example.com	2015-08-08 17:26:06 UTC	509.37	565.5
3	2762	Mykbqx Iodwcgbk	mykbqx.iodwcgbk@example.com	2017-06-29 09:05:13 UTC	695.08	545.52
4	3714	Vfhdkr Vfazkeju	vfhdkr.vfazkeju@example.com	2015-12-15 04:50:57 UTC	685.36	525.5
5	300	Aiet Jwcmzsbs	aiet.jwcmzsbs@example.com	2015-11-26 01:22:03 UTC	444.5	523.76
6	235	Medge Nieves	ac@icloud.couk	2015-04-28 15:30:22 UTC	499.17	522.82
7	3338	Hlyhem Awgcslqd	hlyhem.awgcslqd@example.com	2016-03-15 18:33:50 UTC	462.2	521.86
8	479	Qirpsrzg Canoxoykp	qirpsrzg.canoxoykp@example.c...	2015-07-20 07:38:47 UTC	301.34	517.92
9	456	Ysgrczww Xrfcht	ysgrczww.xrfcht@example.com	2015-07-10 13:34:28 UTC	561.31	517.54
10	3302	Wvnsow Yfbnpmru	wvnsow.yfbnpmru@example.co...	2017-10-14 08:30:48 UTC	426.37	514.93
11	3258	Dxcvsz Fqycuoma	dxcvsz.fqycuoma@example.com	2015-04-05 12:56:12 UTC	452.03	513.59
12	3100	Wpdeav Rkcojarx	wpdeav.rkcojarx@example.com	2015-07-07 19:22:43 UTC	546.52	511.75
13	325	Etwfurw Vfapklpr	etwfurw.vfapklpr@example.com	2016-08-28 18:21:14 UTC	597.98	511.38
14	1952	Drbbrl Mwbvcqyd	drbbrl.mwbvcqyd@example.com	2019-02-27 15:05:57 UTC	433.56	509.09
15	2858	Zrkgim Wrnjhoor	zrkgim.wrnjhoor@example.com	2017-05-08 13:21:54 UTC	582.11	507.95
16	1757	Mxehow Eocgopkb	mxehow.eocgopkb@example.c...	2016-03-12 09:27:24 UTC	667.43	507.44
17	3836	Zphqzs Zoqpdeox	zphqzs.zoqpdeox@example.com	2017-03-21 00:01:11 UTC	451.04	507.27
18	4449	Amudoe Cdwxfgip	amudoe.cdwxfgip@example.com	2018-07-31 19:57:30 UTC	604.87	507.02
19	4850	Ugfdbn Fiotxrrq	ugfdbn.fiotxrrq@example.com	2016-01-20 08:58:20 UTC	471.87	503.77
20	91	Serina Carter	tellus.id.nunc@icloud.com	2015-10-05 09:05:07 UTC	535.06	498.55
21	4957	Lsrlws Nullfbmf	lsrlws.nullfbmf@example.com	2016-08-16 15:02:43 UTC	502.86	497.07
22	3935	Kdrzef Xvwjmava	kdrzef.xvwjmava@example.com	2015-08-05 15:56:22 UTC	184.59	496.75
23	4176	Gevkqt Fueqxrle	gevkqt.fueqxrle@example.com	2015-08-11 08:35:48 UTC	463.43	496.1

Resultados por página: 50 | 1 – 50 de 5000

Nota: para este análisis usamos la tabla 'transactions_clean' porque conserva las fechas reales, imprescindibles para establecer el orden de las compras (las fechas de la tabla optimizada son simuladas).

Nivel 3: Analytics Engineering (Arrays y automatización)

Escenario: Marketing necesita con urgencia un ranking de los productos más vendidos.

Ejercicio 1: Desanidado y aplanado de datos (Unnesting)

Escenario: actualmente, en la tabla de transacciones, los productos vendidos están "escondidos" dentro de un array (una lista de ID). El equipo de ventas quiere analizar qué productos concretos se venden más, pero no puede hacerlo mientras los datos estén comprimidos en una lista. Necesita una tabla detallada (Flat Table) en la que cada fila represente un producto vendido, aunque eso suponga que el ID de la transacción se repita.

🧩 Tarea: crea la tabla dim_transactions_flat desnormalizando la información. Debes "explotar" el array de productos y cruzarlo con el catálogo maestro para obtener los nombres y precios individuales.

⚙️ Pasos técnicos:
Usa CROSS JOIN UNNEST(product_ids) para transformar el array en filas individuales.
Haz un JOIN con la tabla products para obtener el nombre (name) y el precio (price) de cada artículo.
📍 Nota: ten cuidado con los tipos de datos al hacer el JOIN (array de INT64 frente a product_id de tipo STRING).
No agrupes el resultado. Queremos ver el desglose línea a línea.
✅ Resultado esperado: la tabla final debe verse "plana"; una transacción con 3 productos ocupa 3 filas:

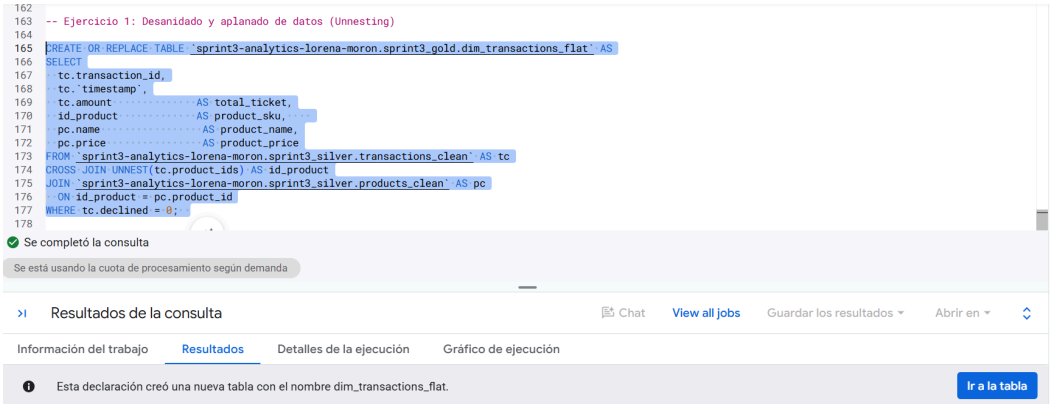
Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

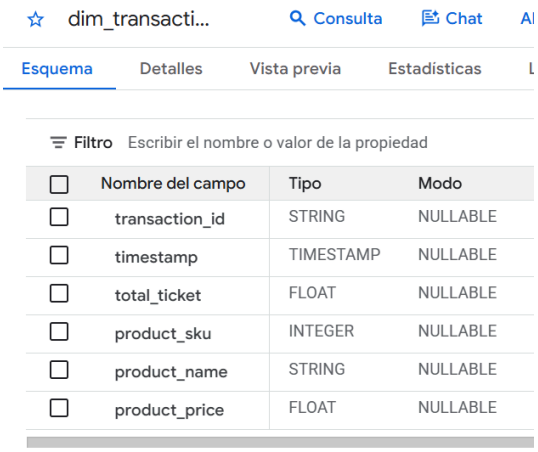
transaction_id	timestamp	total_ticket (global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portátil básico	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratón	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclado	45.50

👉 Entrega:

Creación de la tabla:



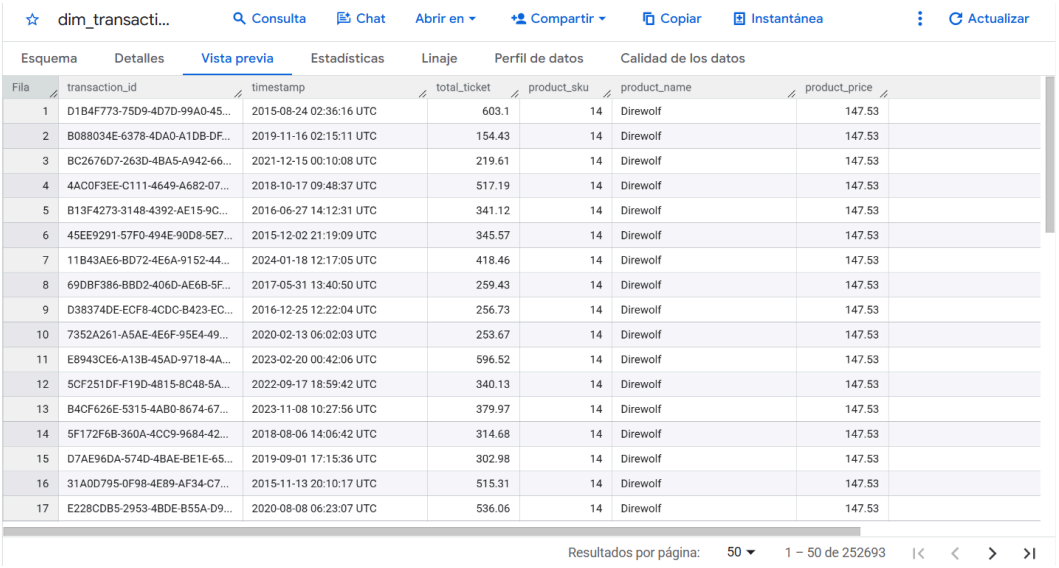
Una captura de pantalla del esquema de la nueva tabla.



Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

Una captura de la vista previa de la tabla en la que se vea que el DDL se ha ejecutado correctamente.



Fila	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price
1	D1B4F773-75D9-4D7D-99A0-45...	2015-08-24 02:36:16 UTC	603.1	14	Direwolf	147.53
2	B088034E-6378-4DA0-A1DB-DF...	2019-11-16 02:15:11 UTC	154.43	14	Direwolf	147.53
3	BC2676D7-263D-4BA5-A942-66...	2021-12-15 00:10:08 UTC	219.61	14	Direwolf	147.53
4	4AC0F3EE-C111-4649-A682-07...	2018-10-17 09:48:37 UTC	517.19	14	Direwolf	147.53
5	B13F4273-3148-4392-AE15-9C...	2016-06-27 14:12:31 UTC	341.12	14	Direwolf	147.53
6	45EE9291-57F0-494E-90D8-5E7...	2015-12-02 21:19:09 UTC	345.57	14	Direwolf	147.53
7	11B43AE6-BD72-4E6A-9152-44...	2024-01-18 12:17:05 UTC	418.46	14	Direwolf	147.53
8	69DBF386-BBD2-406D-AE6B-5F...	2017-05-31 13:40:50 UTC	259.43	14	Direwolf	147.53
9	D38374DE-ECF8-4CDC-B423-EC...	2016-12-25 12:22:04 UTC	256.73	14	Direwolf	147.53
10	7352A261-A5AE-4E6F-95E4-49...	2020-02-13 06:02:03 UTC	253.67	14	Direwolf	147.53
11	E8943CE6-A13B-45AD-9718-4A...	2023-02-20 00:42:06 UTC	596.52	14	Direwolf	147.53
12	5CF251DF-F19D-4815-8C48-5A...	2022-09-17 18:59:42 UTC	340.13	14	Direwolf	147.53
13	B4CF626E-5315-4AB0-8674-67...	2023-11-08 10:27:56 UTC	379.97	14	Direwolf	147.53
14	5F172F6B-360A-4CC9-9684-42...	2018-08-06 14:06:42 UTC	314.68	14	Direwolf	147.53
15	D7AE96DA-574D-4BAE-BE1E-65...	2019-09-01 17:15:36 UTC	302.98	14	Direwolf	147.53
16	31A0D795-0F98-4E89-AF34-C7...	2015-11-13 20:10:17 UTC	515.31	14	Direwolf	147.53
17	E228CDB5-2953-4BDE-B55A-D9...	2020-08-08 06:23:07 UTC	536.06	14	Direwolf	147.53

El resultado tiene 252.693 filas, frente a las 100.000 transacciones de origen: cada transacción se repite tantas veces como productos contiene, con una media de al menos 2,5 productos por transacción.

Nota: No ha sido necesario convertir tipos con CAST, porque tanto product_ids como product_id ya son INTEGER tras la limpieza de la capa Silver.

Ejercicio 2: El ranking de ventas (agregación simple)

Escenario: ahora que has preparado la infraestructura y dispones de la tabla dim_transactions_flat (en la que cada fila equivale a un producto vendido), generar informes analíticos es trivial. Ya no necesitas funciones complejas de desanidado para responder a preguntas básicas de negocio.

🧩 Tarea: genera el Top 5 de productos más vendidos en la historia de la compañía.

🛠️ Método: consulta directamente tu nueva tabla desnormalizada (dim_transactions_flat). Al ser una tabla plana, solo necesitas una agrupación estándar (GROUP BY) por nombre de producto y un recuento (COUNT) de las filas.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

👉 Entrega:

El código SQL que genera el ranking.
Una captura del resultado con los 5 productos ganadores y las unidades vendidas de cada uno.

```sql
-- Ejercicio 2: El ranking de ventas (agregación simple)

SELECT dtf.product_sku,
  dtf.product_name,
  COUNT(dtf.product_sku) AS q_sales
FROM `sprint3-analytics-lorena-moron.sprint3_gold.dim_transactions_flat` AS dtf
GROUP By dtf.product_sku, dtf.product_name
ORDER BY q_sales DESC
LIMIT 5;
```

Esta consulta procesará 5.97 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta | Chat | View all jobs | Guardar los resultados | Abrir en

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	product_sku	product_name	q_sales
1	52	riverlands the duel	2642
2	29	Tully maester Tarly	2627
3	21	duel Direwolf	2603
4	16	the duel warden	2602
5	33	duel warden	2593

Resultados por página: 50 | 1 – 5 de 5

Ejercicio 3: Automatización del pipeline y visualización

Escenario: el informe estático del ejercicio anterior ha gustado mucho, pero el Director de Marketing necesita profesionalizarlo para la toma de decisiones diaria. Tiene dos requisitos urgentes:

Lógica centralizada: quiere ver desglosado el precio con IVA (21 %). Sin embargo, como el impuesto puede cambiar según el país o el año, te prohíbe escribir el cálculo precio * 1.21 directamente en el código (hardcoded). Debes usar una función reutilizable.
Actualidad de los datos: el panel debe actualizarse automáticamente cada mañana a las 07:00, antes de la reunión diaria de ventas.

🧩 Tarea: debes evolucionar la tabla dim_transactions_flat para que incluya el cálculo de impuestos y automatizar su regeneración.

⚙️ Pasos técnicos:
Funciones definidas por el usuario (UDF): crea una función SQL persistente llamada calculate_tax(amount) que reciba un valor numérico y devuelva el resultado aplicando el 21 % de impuesto.
Integración y orquestación:
Modifica el código de creación de la tabla (del Ejercicio 1) para que use tu nueva función calculate_tax y genere una columna nueva: product_price_tax_inc.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

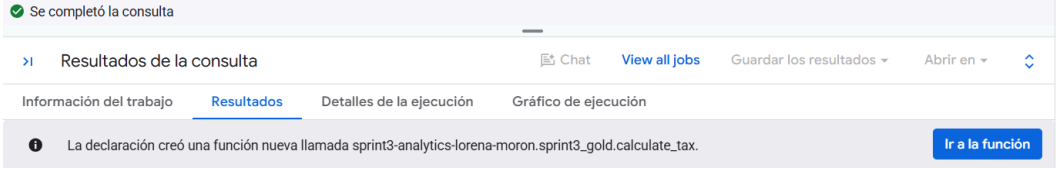
Configura una Scheduled Query (consulta programada) en BigQuery para que esta sentencia (CTAS) se ejecute automáticamente cada día a las 07:00.
Visualización (BI): conecta Looker Studio a tu tabla dim_transactions_flat y crea un dashboard llamado "Monitor de Rendimiento de Ventas".

👉 Entrega:

El código SQL de la UDF.

```sql
-- Ejercicio 3: Automatización del pipeline y visualización
-- Paso 1: Creación de UDF

CREATE OR REPLACE FUNCTION `sprint3-analytics-lorena-moron.sprint3_gold.calculate_tax`(amount FLOAT64)
RETURNS FLOAT64
AS (
  amount * 1.21
);
```



El código SQL actualizado de creación de la tabla.

```sql
-- Paso 2: Modificación de la tabla dim_transactions_flat

CREATE OR REPLACE TABLE `sprint3-analytics-lorena-moron.sprint3_gold.dim_transactions_flat` AS
SELECT
  tc.transaction_id,
  tc.`timestamp`,
  tc.amount              AS total_ticket,
  id_product             AS product_sku,
  pc.name                AS product_name,
  pc.price               AS product_price,
  ROUND(`sprint3-analytics-lorena-moron.sprint3_gold.calculate_tax`(pc.price) - pc.price,2) AS product_tax_amount,
  ROUND(`sprint3-analytics-lorena-moron.sprint3_gold.calculate_tax`(pc.price),2)            AS product_price_tax_inc
FROM `sprint3-analytics-lorena-moron.sprint3_silver.transactions_clean` AS tc
CROSS JOIN UNNEST(tc.product_ids) AS id_product
JOIN `sprint3-analytics-lorena-moron.sprint3_silver.products_clean` AS pc
  ON id_product = pc.product_id
WHERE tc.declined = 0;
```



Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

dim_transacti... | Consulta | Chat | Abrir en | Compartir | Copiar | Instantánea | Borrar | Actualizar

Esquema | Detalles | Vista previa | Estadísticas | Linaje | Perfil de datos | Calidad de los datos

Fila	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price	product_tax_...	product_pric...
1	D88B7596-AD70-43C2-B479-65...	2017-04-26 09:22:19 UTC	147.53	14	Direwolf	147.53	30.98	178.51
2	8D336751-359C-47F1-B325-DE...	2022-11-16 00:33:13 UTC	443.37	14	Direwolf	147.53	30.98	178.51
3	20AE5316-80EB-43BF-95B2-A0...	2018-05-14 18:16:56 UTC	470.53	14	Direwolf	147.53	30.98	178.51
4	B910CED7-F256-4943-92ED-D8...	2016-02-03 05:43:10 UTC	340.13	14	Direwolf	147.53	30.98	178.51
5	D5E2F383-E2D3-46AE-A4DA-16...	2017-03-30 16:00:09 UTC	250.58	14	Direwolf	147.53	30.98	178.51
6	982B1F3C-C887-4907-BB1E-CB...	2024-01-25 07:47:26 UTC	343.15	14	Direwolf	147.53	30.98	178.51
7	B435A89E-B323-40D1-AD77-76...	2024-10-28 16:33:13 UTC	343.15	14	Direwolf	147.53	30.98	178.51
8	7288E620-DED4-4B26-BDF0-F4...	2015-07-03 03:09:55 UTC	147.53	14	Direwolf	147.53	30.98	178.51
9	0A654DB4-1663-43C2-9E58-34...	2017-12-10 18:58:26 UTC	276.01	14	Direwolf	147.53	30.98	178.51
10	6C5A8AD5-1B67-46C1-BE0C-E7...	2015-12-24 14:16:09 UTC	205.25	14	Direwolf	147.53	30.98	178.51
11	477D10C2-A9AC-4FBB-9B60-14...	2017-08-31 17:00:52 UTC	220.84	14	Direwolf	147.53	30.98	178.51
12	5FF15AED-B02D-4573-A079-79...	2015-05-26 04:23:29 UTC	381.27	14	Direwolf	147.53	30.98	178.51
13	A0AB8AD5-4BCE-4CFA-BFC4-6F...	2021-05-24 00:41:26 UTC	589.7	14	Direwolf	147.53	30.98	178.51
14	F0728750-9B0E-46F3-BC32-6A...	2018-12-21 12:18:42 UTC	633.31	14	Direwolf	147.53	30.98	178.51
15	270B461E-ADA3-4E92-92FA-E5...	2019-02-20 09:20:12 UTC	412.31	14	Direwolf	147.53	30.98	178.51
16	78489ECE-8CBB-47F0-A4D1-63...	2021-09-14 20:56:59 UTC	543.45	14	Direwolf	147.53	30.98	178.51
17	D9433375-6753-44E6-89C3-66...	2024-10-22 03:56:42 UTC	469.42	14	Direwolf	147.53	30.98	178.51
18	C13D5AC6-51D5-49BB-8CB1-C...	2021-02-27 06:36:14 UTC	219.01	14	Direwolf	147.53	30.98	178.51
19	F9ACF0E1-0B13-4267-9637-EB...	2023-01-28 02:27:32 UTC	147.53	14	Direwolf	147.53	30.98	178.51
20	2AA12C00-0EAD-4BB9-A4EF-21...	2020-03-22 20:44:22 UTC	444.28	14	Direwolf	147.53	30.98	178.51

Resultados por página: 50 | 1 – 50 de 252693

Nota: Las columnas con IVA se redondean a 2 decimales al crear la tabla porque representan importes monetarios reales (el precio que paga el cliente por unidad), que en la práctica existen al céntimo. El IVA se calcula a partir del precio con IVA ya redondeado, para que el desglose cuadre siempre: precio base + IVA = precio con IVA. En cambio, las métricas intermedias que se reutilizan en otros cálculos (como las ventas diarias de la vista materializada) se guardan sin redondear y solo se redondean en la presentación final.

Configuración de una Scheduled Query:

Exportar como consulta programada

Configuración válida.

Detalles y programación

Nombre de la consulta programada *
daily_refresh_dim_transactions_flat

Opciones de programación

Frecuencia de repetición *
Días

A la(s) *
5:00 UTC

Comenzar ahora | Comenzar a la hora definida

Fecha de inicio y hora de ejecución
15/10/26, 11:51 a.m.

Sin hora de finalización programada
Programar una hora de finalización

Fecha de finalización

Programado para ejecutarse todos los días a las 5:00 UTC a partir de ahora.

Destino de los resultados de la consulta

Establecer una tabla de destino para los resultados de la consulta

Conjunto de datos

Table Id

Campo de partición de la tabla de destino

Preferencia de escritura para la tabla de destino

Agrega a la tabla
Reemplaza la tabla

Selección automática de ubicación

Tipo de ubicación

Región
Especifica una región para colocar tus conjuntos de datos con otros servicios de Google Cloud.

Multirregión
Permite que BigQuery seleccione una región de un grupo para alcanzar límites de cuota más altos.

Región

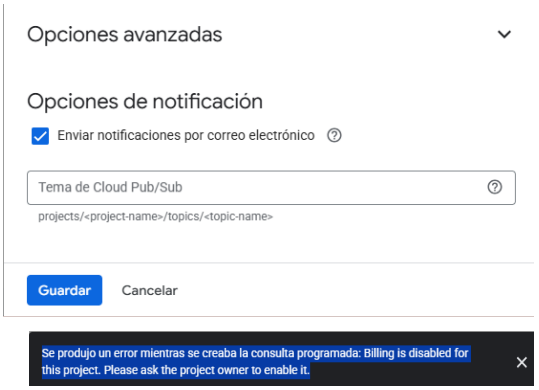
Cuenta de servicio

Cuenta de servicio

Configuración predeterminada para usar la credencial de usuario solicitante si no se selecciona ninguna cuenta de servicio

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón



Nota: Se ha configurado la consulta programada daily_refresh_dim_transactions_flat con ejecución diaria a las 05:00 UTC (07:00 en Madrid en horario de verano y 06:00 en horario de invierno), sin tabla de destino, ya que la propia sentencia CREATE OR REPLACE TABLE define dónde se guarda el resultado.

El formulario validó la configuración correctamente, pero al guardarla BigQuery devolvió el error "Billing is disabled for this project": las consultas programadas dependen del Servicio de Transferencia de Datos, que el entorno Sandbox no admite sin facturación, tal como indica el enunciado. Se adjuntan capturas de la configuración y del error. En un proyecto con facturación habilitada, bastaría con guardar esta configuración para que la tabla se regenerase automáticamente cada mañana.

Un enlace al gráfico en Looker Studio.

📌 Propuesta de dashboard: "Monitor de Rendimiento de Ventas"

Configura una página con 3 secciones clave:

1. Los "Big Numbers" (KPI generales)
En la parte superior, coloca tres tarjetas de resultados (Scorecards) que den contexto inmediato:

KPI 1: Ingresos totales (con IVA)
Métrica: SUM(product_price_tax_inc)
Por qué: muestra el volumen real de dinero que entra.
KPI 2: Transacciones únicas (pedidos)
Métrica: COUNT_DISTINCT(transaction_id)
KPI 3: Ticket medio (en el original aparece repetido el título "Transacciones únicas", pero por la fórmula se trata del ticket medio)
Campo calculado: crea en Looker el campo SUM(product_price_tax_inc) / COUNT_DISTINCT(transaction_id)

2. Evolución temporal (gráfico de series temporales)
Debajo de los KPI, un gráfico de líneas que ocupe todo el ancho:

Dimensión: timestamp (desglosado por fecha).
Métrica: SUM(product_price_tax_inc).
Línea de referencia (opcional): añade una media constante o una línea de tendencia.
Valor: permite a Marketing ver si las ventas suben o bajan día a día, o si hubo un pico por alguna campaña concreta.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

3. El "Top Productos" enriquecido (tabla con mapa de calor)
Olvídate del gráfico de barras simple. Usa una tabla con barras incrustadas o con mapa de calor:

Dimensión: product_name.
Métrica 1: COUNT(*) → renómbrala como "Unidades vendidas".
Métrica 2: SUM(product_price_tax_inc) → renómbrala como "Ingresos totales".
Métrica 3: SUM(product_price_tax_inc) - SUM(product_unit_price) → renómbrala como "IVA recaudado" (esto demuestra el valor de tu UDF).
Estilo: en la configuración de estilo de la tabla, activa la opción "Mostrar número como: Barra" o "Mapa de calor" para la columna de ingresos.

Sprint 4. Ingeniería avanzada de BigQuery y Analítica

Realizado por: Lorena Morón

Enlace para acceder al dashboard en Looker Studio:

https://datastudio.google.com/reporting/753e8adc-bb09-4f66-840c-6c305b5159c1

